- Saeko API Documentation
- Authentication
- REST Guide
- Resources
- /api/v1/core
  - School
  - Term
  - Program
  - Group
  - Subject
  - Course
  - Student
  - Employee
  - Contact
- /api/v1/location
  - Address
  - State
  - Country
- /api/v1/accounting
  - Scholarship
  - ScholarshipType
- /api/v1/admissions
  - AdCampaign
- /api/v1/grading
  - Activity
  - Grade
  - ExtraordinaryGrade
- /api/v1/scheduling
  - GateAccessLog
- /api/v1/provisioning
  - User

Saeko API Documentation

Welcome to the Saeko API documentation! With this guide you will be able to get around the basics our REST API and the core models of Saeko.

Authentication

OAUTH2 - JWT FOR DOMAIN-WIDE SERVICE ACCOUNTS

Saeko API can authorize enterprise applications to access resources owned by a domain administrator, without requiring explicit consent from a user. With a private key a third-party application can delegate authority to any user inside the domain.

CONCEPTS

- **Domain**: In Saeko every customer owns a single domain. All data owned by the customer is available only inside of the domain.
- **Client**: (My Application Server) The third-party application that has access to the domain.
- **Service Account**: a private key used by the client to access the domain. These keys have an expiration date and should be renewed.
- **User Account**: an actual account created inside of Saeko; represented by an email and password.
- **Authentication Server**: Saeko has a single server that manage the OAuth2 protocol.
- **API Server**: Saeko has multiple servers hosting one or more domains.
- **JSON Web Token**: (JWT) is an open standard (RFC 7519). In this implementation Saeko creates an asymmetric RSA key pairs: a private key secured by the client (signs a JSON message), and a public key stored in Saeko (verifies the integrity of the JSON message). More information at https://jwt.io.

REQUIREMENTS

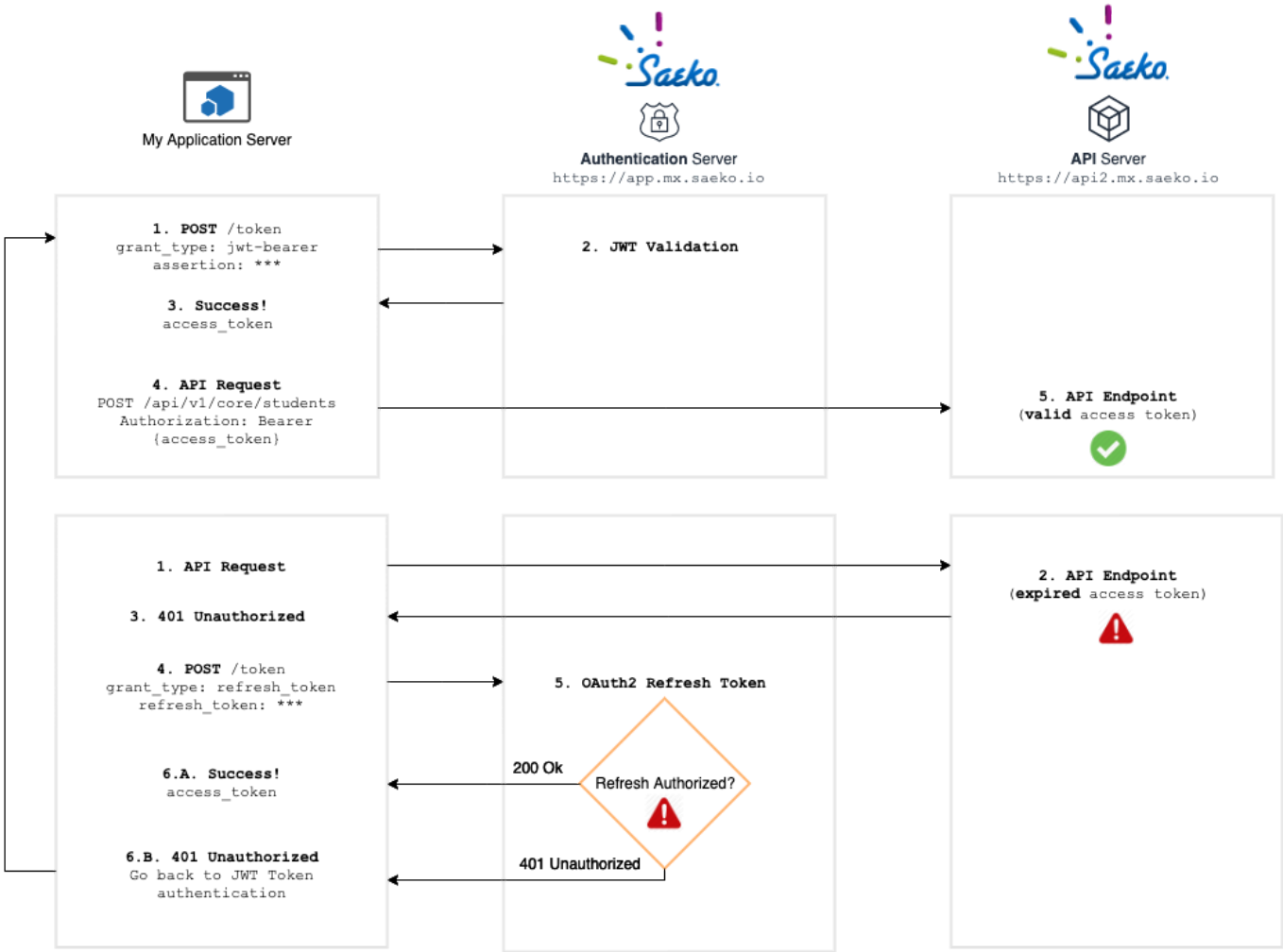
For the client to access a domain it needs:

- 1) Service Account credentials: (ask our support team for credentials)

```
service_account:
  auth_url: "https://..."
  api_url: "https://..."
  client_id: "..."
  private_key_id: "..."
  private_key: "-----BEGIN RSA PRIVATE KEY-----\n...\n-----END RSA PRIVATE KEY-----"
  expires_at: "2021-01-01T00:00:00Z"
```

- 2) A user account to delegate authority to. Only the email is required.
  - This account must be created from the Administrator Portal in Saeko.
  - The permissions assigned to this user will be enforced when accessing the API with the Service Account.

AUTHENTICATION FLOW



HOW TO CREATE ACCESS TOKEN (DELEGATE AUTHORITY)

Request a token to the OAuth2 server with a JSON Web Token (JWT).

- Use the following values to build the JWT payload:

```
const payload = {
  iss: `${client_id}`,
  sub: `${user_email}`,
  scope: "admin",
  iat: `${timestamp_epoch}`,
  jti: MD5(`${private_key_id}:${timestamp_epoch}`)
}
```

- `client_id`: from the service account.
- `user_email`: the email of the user you want to delegate access to.
- `private_key_id`: from the service account.
- `timestamp_epoch`: the server's Epoch time.

- Encode the JWT with your private key using RS256.

```
const jwt = JWS.sign(null, { 'typ': 'JWT', 'alg': 'RS256'}, payload, private_key);
```

- Build the request and use the encoded JWT in the body with `Content-Type: application/json`:

```
const response = await fetch(auth_url, {
  method: 'POST',

  headers: {
    'Content-Type': 'application/json'
  },

  body: JSON.stringify({
    "grant_type": "urn:ietf:params:oauth:grant-type:jwt-bearer",
    "assertion": `${jwt}`
  })
})
```

The authentication server will validate the JWT and create an access token for the delegated user.

- If the delegated user does not belong to the domain, or is an invalid user, then a `401 Unauthorized` error code will be returned.

🚨 Even when the examples above are in JavaScript, NEVER use this authentication flow from the user's browser. The private key must always be stored in a secure environment.

This is and example of the body response:

```
{
  "access_token": "***",
  "token_type": "bearer",
  "expires_in": 3600,
  "refresh_token": "***",
  "scope": "admin",
  "created_at": 1577836800
}
```

- `expired_in` must be used to refresh the token before expiration.
- `access_token` must be used in all future API requests in the headers:

```
{ 'Authorization': `Bearer ${access_token}` }
```

HOW TO REFRESH THE ACCESS TOKEN

Just before the token expires or when receiving a `401 Unauthorized` response from the API, try refreshing the access token.

Reasons for receiving a `401 Unauthorized` response from the API:

- Token expired; most common.
- The user doesn't exists any more, or has been downgraded/deactivated.
- The API has denied access to the token/user.

To refresh the token make an OAuth2 request to `POST /oauth/token` with the body:

```
{
  "refresh_token": "***",
  "scope":"admin",
  "grant_type":"refresh_token"
}
```

Replace the access token and resume the API requests.

When the refresh token request replies with `401 Unauthorized`, then the refresh token has been revoked and the client must invalidate and delete the access token object entirely.

REST Guide

The Saeko API is organized by resources following a REST Architecture design:

Resources in Saeko can be fetched, created, updated, and deleted using:

- an HTTP Method
- a URL that represents the resource and,
- a well defined resource/payload object

HTTP HEADERS

- `Authentication: Bearer ...` - Use the access token to authenticate the request.
- `Content-Type: application/json; charset=utf-8` - When using POST or PUT the body must always be a JSON object.

HTTP QUERY PARAMETERS

Resources are usually simple objects, but for efficiency and practicality you may find yourself in these situations:

- Need extra fields or objects embed inside of resources to avoid N+1 requests.
  - Use either `fields` or `include_fields`
  - `fields` - to specify exactly which fields you need from a resource.
  - `include_fields` - to request additional attributes; those marked with extra.
  - set the value in the form: `?include_fields=field_a,field_b,nested_object(field_x,field_y)`.
  - use only in endpoints that explicitly specifies support for fields.
- Filter, paginate, and sort a long list of data; sometime imposed by the API.
  - `filters` - filter a GET request.
    - set the value in the form of `?filters=field_a=1,2;boolean_field;association_object(field_a=1)`
    - separate multiple filters with `;`,
    - you can filter by multiple values, separate each one with `,`; e.g. `?filters=status=1,2`
    - boolean filters don't need values; e.g. `?filters=active`
  - `limit` - activate pagination; e.g. `?limit=30`
    - `offset` can help move the response to the next set of records; e.g. `?limit=20&offset=20`
    - use this parameters only for endpoints that explicitly specifies support for pagination
    - some endpoints have pagination enforced.
  - `sort_by` - sort results; e.g. `?sort_by=created_at:desc`
    - `:asc` and `:desc` are the only valid order values
    - the order is optional, as the default is `:asc`; e.g. `?sort_by=created_at`
    - use this query parameter when doing pagination to get consistent results.

Every query value MUST be URL safe encoded, e.g. `?filters=status%3D1%2C2`.
For presentation purpose the examples in this documentation are not URL encoded.

HTTP RESPONSE CODES

- `200 OK` - Success.
- `204 No Content` - A DELETE request successfully deleted the resource.
- `400 Bad Request` - The request body or the path is malformed or inexistent.
- `401 Unauthorized` - The access token has expired or is invalid.
- `403 Forbidden` - The request is not accessible by the current user.
- `404 Not Found` - The requested resource, or context, does not exist.
- `422 Unprocessable Entity` - When using POST/PUT and there was an invalid field in the payload. Usually the response body will contain a simple explanation.

COMMON ERROR DESCRIPTIONS

Error descriptions can be found in a JSON formatted response, e.g.:

```
{ "error_description": "Invalid Path" }
```

Example of common error descriptions:

- `Invalid Path` - The request URL PATH is invalid or inexistent. Example of invalid paths:
  - `/api/v1/core/terms//students` (the `term_id` is missing)
  - `/api/v0/core/user` (invalid API version)
- `Malformed Payload` - Happens when the request has a JSON payload that is malformed, e.g.:
  - `{ "students": { "name": "Name", } }` (JSON object is invalid because there is an extra ",")
- `Parameter required: ...`- The payload MUST contain the specified value to continue.
  - Spanish: `Parámetro obligatorio: ...`
- `Invalid parameter: ...` - The payload contains a specific invalid value that should be fixed accordingly the resource documentation.
  - Spanish: `Parámetro inválido: ...`
- `Unprocessable entity: ...` - The payload contains general invalid values that can not be processed.
  - Spanish: `Dato de entrada inválido: ...`
- `Resource not found: ...` - May be an invalid context, or a resource in the payload that does not exist.
  - Spanish: `Recurso no encontrado:`
- `Bad request` - A reason is not specified, but there is something wrong with the client's request: either in the payload, or the PATH.
  - Spanish: `Solicitud inválida`
- `Access forbidden` - User is not allowed to perform this request until permissions are granted.
  - Spanish: `Acceso restringido`
- `User unauthorized` - The user credentials, access token in the Authentication header, are expired or inexistent. Renewing the access token should usually resolve this error.
  - Spanish: `Usuario no autorizado`
- `Server error` - Unspecified error in the server side.
  - Spanish: `Error en el servidor`

- `Request not allowed` - The request action (HTTP Method) cannot be executed to the resource or the context. This error can appear when updating a protected/locked resource.
    - Spanish: `Solicitud no permitida`

REST METHODS - EXAMPLES

HTTP Method	Path	Used for
GET	/api/v1/core/schools	List schools available to the user
GET	/api/v1/core/schools/:id	Show a single school object by :id
POST	/api/v1/core/schools	Create a new school
PUT	/api/v1/core/schools/:id	Update an existing school by :id
DELETE	/api/v1/core/schools/:id	Delete a school permanently by :id

REST METHODS - EXAMPLES (WITH CONTEXT)

HTTP Method	Path	Used for
GET	/api/v1/core/schools/:school_id/terms	List terms filtered by :school_id
GET	/api/v1/core/terms/:id	Show a single term object by :id
POST	/api/v1/core/schools/:school_id/terms	Create a new term in the :school_id
PUT	/api/v1/core/terms/:id	Update an existing term by :id
DELETE	/api/v1/core/terms/:id	Delete a term permanently by :id

RESOURCE PAYLOAD

When doing a POST or PUT to **create** or **update** a resource you must always send a JSON object with the resource name in `snake_case` and singular:

E.g. to create a new student, the payload will be:

```
{
    "student": {
        "name": ""
        // ...
    }
}
```

RESOURCE PAYLOAD - MULTI

There are a few resources that support more than one resource to either create or update. In this case the request will be POST and the payload will have the resource name in plural.

E.g. to save multiple periods in a school term:

`POST /api/v1/core/terms/:term_id/periods`

```
{
    "periods": [
    {
        // use ID to update existing periods
        "id": 1,

        // updates the name for the existing period
        "name": "Period 1"
        },
        {
            // creates a new period with this name
            "name": "Period 2"
        }
    ]
}
```

Never use `id` on the resource payload then sending a singular resource object:

```
{
    "period": {
      // THIS IS INVALID:
      "id": 1,

      "name": "Period 1"
  }
}
```

Resources

API STRUCTURE

Resources in SaekoAPI are structured in different **namespaces** and **contexts** that are represented by consistent URL Paths.

- **namespace**: is a group of resources that make sense together.
  - A namespace will have the form `/api/v1/{namespace}`
  - E.g. resources related to a geographical locations are found in the namespace `/api/v1/location`: Address, State and Country.
- **context**: is a parent resource that in a way provides context to other related resources.
  - A context path will have the form `/api/v1/{namespace}/{context-resource-name}/{context-resource-id}/{resource}`
  - E.g. To create a new student you must provide a Term as context: `POST /api/v1/core/terms/:term_id/students` (`:term_id` should be replaced with an actual ID).
  - Contexts are used by the API con control user access to resources as well as building requests that make more sense to the client.
  - ⚠️ If the context is invalid or inexistent the request will usually respond with `404 Not Found`.

NOTES LEGEND

- required
  - when creating a new object with POST: field value must be present and valid.
  - when updating an existing object with PUT: field value can't be nullified.
- read-only
  - fields only available on GET requests.
- write-only
  - fields can only be used in POST or PUT requests.
  - usually these fields are friendly substitutions to more complex data structures. E.g. `student.mobile_phone` will find in the `phones` array a record with `type = MOBILE` and update the `phone_number`, or create a new phone record.
- embed
  - fields that are objects and can be used in a POST/PUT request to save association records.
  - fields that are available in a GET request as embed associations.
- extra
  - fields that are not available by default in a GET request, and must be explicitly requested using `?include_fields=my_extra_field`. See HTTP Query Parameters.
- association
  - can be belongs-to, has-one or has-many.
  - if the field name has a suffix `_id`, then this field must be the ID of associated object.
  - if combined with embed then the associated object can be found embedded in the payload; response (read-only), request (write-only) or both.

/api/v1/core

School

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
name	string		
logo_url	string		

LIST OF ENDPOINTS

- GET /api/v1/core/schools

  List all schools available to the current user.

  Allowed Users: adminapplicant
- GET /api/v1/core/schools/id

  Show school by ID.

  Allowed Users: student

Term

DESCRIPTION

A Term in Saeko represents a School Year or Semester. Almost every process in Saeko are related to a single Term, e.g.: enrollments, groups, courses, payment plans, etc.

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
name	string		
term_type	integer	`Bimester: 1` `Trimester: 2` `Quarter: 3` `Semester: 4` `Year: 5`	
begins_at	date	`YYY-MM-DD` - First day of class.	
ends_at	date	`YYY-MM-DD` - Last day of class.	
is_current	boolean	`TRUE` when the current date is between the Term date range (`begins_at` - `ends_at`)	read-only
periods	objects	Array of simple objects describing term periods.	read-only

LIST OF ENDPOINTS

- GET /api/v1/core/schools/school_id/terms

  List of terms filtered by a school-id

  Allowed Users: admin

Program

DESCRIPTION

A Program defines the school curricula. Students are usually enrolled every term on a specific program.

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
name	string		required
terms_quantity	integer	Total number of terms required to complete the program. E.g. a Primary plan with 6 years: `terms_quantity: 6`.	required
term_type	integer	See Term#term_type.	required
level	integer	`MAESTRIA = 1` `LICENCIATURA = 2` `BACHILLERATO = 3` `SECUNDARIA = 4` `PRIMARIA = 5` `PREESCOLAR = 6` `NIVELES = 7` `ESPECIALIDAD = 8` `DOCTORADO = 9` `MATERNAL = 10` `DIPLOMADO = 11` `CURSO = 12` `CURSO_TALLER = 13` `SEMINARIO = 14` `TALLER = 15`	required
school_ids	array	It's an array for schools who has blabalba	required
plan	string	Text in reports eg. "2020",	
official_name	string	eg. "Lic. en Administración",	
grade_by_student	boolean	Helps to assess the subject by students like primary school eg. false,	
incorporation	string	Text in reports eg. "Marzo 2020",	
agreement	string	Text in reports eg. "A00139",	
header	string	Text in reports eg. "",	
rvoe	string	Number of the program for private schools eg. "LOE132019"	

LIST OF ENDPOINTS

- GET /api/v1/core/schools/school_id/programs

  List of programs filtered by school id

  Allowed Users: adminprofessorstudentapplicantsignee
- GET /api/v1/core/terms/term_id/programs

  List of Programs filtered by Term.
  Terms and Programs are not directly associated, but the attributes `term.school_id` and `term.term_type` can filter programs.
  Useful when you only need programs compatible with a specific term.
  E.g. after fetching programs by term_id, you can then create a student's `initial_enrollment` with a `program_id` that is compatible with the `term_id`.

  Allowed Users: adminsignee
- GET /api/v1/core/programs/id

  Show program by ID.

  Allowed Users: adminprofressorstudentapplicant
- POST /api/v1/core/programs

Add a program

Allowed Users: admin

Group

DESCRIPTION

Students can be enrolled every school year into one and only one group. A group has a set of courses related to a single program. Students can enroll courses from the enrolled group, but also can enroll courses from other groups.

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
name	string		required
grade_level	integer	Grade level must be a number between 1 and program.terms_quantity.	required
program_id	integer	The program must have the same term_type as the term.term_type when creating the group.	required
shift	integer	`MATUTINO = 1` `VESPERTINO = 2` `MIXTO = 3` `NOCTURNO = 4` `SABATUNO = 5` `FIN_DE_SEMANA = 6` `DISCONTINUO = 7`	required
max_students	integer	When this field is greater than 0, then the number of students that can enroll to this group will be limited.	
default_room	integer	When creating courses in the group, this default room can be used.	

status	integer	Only available on app.saeko.io:
		`ST_CREADO = 1` (default)
		`ST_AUTORIZADO = 2`
		`ST_RECHAZADO = 3`	

LIST OF ENDPOINTS

- GET /api/v1/core/terns/term_id/groups

  List of groups by term id

  Allowed Users: admin
- GET /api/v1/core/groups/id

  Group details, can include embed courses

  Allowed Users: admin
- POST /api/v1/core/terms/term_id/groups

  Create group(s) in the specific term id. This endpoint supports multiple groups, which can be used not only to create new groups, but also to update existing groups.

  Allowed Users: admin

  Example:

```
{
  "groups": [
      {
          // update existing group
          "id": 1,
          "name": "1A",
          "grade_level": 1,
          "program_id": 1,
          "shift": 1,
          "max_students": 40,
          "default_room": "Aula 1",
          "status": 2
      },
      {
          // create new group
          "name": "1B",
          "grade_level": 1,
          "program_id": 1,
          "shift": 1,
          "max_students": 40,
          "default_room": "Aula 2",
          "status": 2
      }
  ]
}
```

Subject

DESCRIPTION

A Subject is part of a program or a multiple programs. Students are usually enrolled every term on a specific group of subjects.

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
name	string	Name of the subject, it will appear in the report card of the students eg. "Ingles II"	required
code	string	Special code, it will appear in the report card of the students eg. "II02"	required
min_passing_grade	integer	Minimun score to pass the subject. E.g. 7 is the minimum score for a student to pass the subject.	required
grade_level	integer	The grade where this subject belong and it will be able to enroll by a student E.g. Spanish I: `grade_level: 1`	required
program_ids	array	An array of programs which are related to the subject	required
credits	decimal	10.5, 0..999	
order	integer	Helps to order subjects in reports 1..10	
active	boolean	Active subjects will appear when you are creating groups and classes by efault TRUE on new subjects	
absences_limit	integer	Absence limit prints a report who are able to present exam in class 1...10	
hours	integer	Works when subjects are connected with other subjects E.g. Maths has Theory (50) and Practice (50). Theory `hours:50`	
skip_average	boolean	Counts for the GPA(promedio) E.g. false	
discipline_field	integer		
required_subject_id	integer	Must be a subject that belongs to the same program. `required_subject_id:2162`	
propaedeutic_area_id	integer	1	

LIST OF ENDPOINTS

- GET /api/v1/core/programs/program_id/subjects

  List of subjects filtered by program id

  Allowed Users: admin
- GET /api/v1/core/subjects/id

  Show subject by ID.

  Allowed Users: admin
- POST /api/v1/core/subjects

  Create a new subject

  Allowed Users: admin

</> Course

DESCRIPTION

Courses are subjects that can be enrolled by students in a school year (term). Every course will belong to a group, therefor to a term and program.

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
room	string		
attendance_enabled	boolean		
absences_limit	integer	Students who have more absences more than the absences limit, it will appear in a list with a special badge.	
professor_id	Professor		belongs-to
subject_id	Subject		required belongs-to
is_group_course	boolean	Group courses are courses without subjects, that can be used to make roll calls on the group instead of a specific course.	

LIST OF ENDPOINTS

- GET /api/v1/core/terms/term_id/courses

List of courses by term id. Pagination is enforced with a max limit of 100.

Allowed Users: admin

- GET /api/v1/core/groups/group_id/courses

  List of courses by group id

  Allowed Users: admin

- POST /api/v1/core/groups/group_id/courses

  Create course(s) in the specific group. This endpoint supports multiple courses, which can be used not only to create new courses, but also to update existing courses.

  Allowed Users: admin

  Example:

```
{
    "courses": [
    {

        "room": "Aula 1",
        "attendance_enabled": true,
        "professor_id": 1,
        "subject_id": 1
    },
    {
        // it's possible to update an existing course using the ID
        // it MUST belong to the same context group
        "id": 1,
        "room": "Aula 2"
    }]
}
```

Student

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
name	string	First and middle name are concatenated in this attribute.	required
first_surname	string		required write-only
second_surname	string		write-only
surnames	array	First and second surnames are separated in an array within this attribute.	read-only
student_id	string boolean	This is the Student's permanent enrollment ID (a.k.a "Matrícula" in spanish). Use the value `student_id: true` in order to: • auto-generate a new student ID using a preset format. • The preset format must be configured from the Saeko Admin Portal; if not, this attribute will be ignored. • The current `student_id` must be `null` before auto-generating a new id or an HTTP `422` Error will be responded.	
personal_email	string		
gender	integer	`MALE: 1` - `FEMALE: 2`	required
curp	string	Defined by the agency RENAPO. Limit: exact 18 chars. **Field available in this priority:** • GET List: extra field. • Admin User: available. • Other User: extra field.	
rfc	string	Registro Federal de Contribuyentes. Limit: 12 to 13 chars. **Field available in this priority:** • GET List: not available. • Admin User: available. • Other User: extra field.	
birthplace	string	**Field available in this priority:** • GET List: not available. • Admin User: available. • Other User: extra field.	

Attribute	Type	Description	Notes
birthdate	date	Format: `YYYY-MM-DD` **Field available in this priority:** • GET List: not available. • Admin User: available. • Other User: extra field.	
phones	array	All phones are embed in this array using the types: `MOBILE = 1` `HOME = 2` `OFFICE = 3` `OTHER = 4` `FAX = 5` **Field available in this priority:** • GET List: not available. • Admin User: available. • Other User: extra field.	read-only embed
phone	string	Limit: 0 - 15 chars. `type: HOME`	write-only
mobile_phone	string	Limit: 0 - 15 chars. `type: MOBILE`	write-only
fax_phone	string	Limit: 0 - 15 chars. `type: FAX`	write-only
other_phone	string	Limit: 0 - 15 chars. `type: OTHER`	write-only
comments	string	**Field available in this priority:** • GET List: not available. • Admin User: available. • Other User: not available.	
invoice_receiver	object	For accounting purposes, the student may optionally have one default InvoiceReceiver association. **Field available in this priority:** • GET List: not available. • Admin User: available. • Other User: not available.	write-only embed
invoice_receiver.rfc	string	Must be 12 or 13 characters long. Required only if you set the `invoice_receiver` object. Saeko avoids duplicate InvoiceReceivers by using the RFC as a unique identifier.	
invoice_receiver.email	string	If the RFC already exists in the database, you will end up replacing the email of the invoice receiver.	
addresses	Address	All student addresses are available in this array **Field available in this priority:** • GET List: not available. • Admin User: available. • Other User: extra field.	read-only extra has-many
home_address	Address	Use this attribute to create/update a student's home address. If the student has multiple addresses only the first one to have a `type: HOME` will be updated, or a new home address will be appended. The `home_address.type` field is ignored.	write-only embed has-one
initial_enrollment	Enrollment	When creating a new student an enrollment will be automatically created. Set this object to specify other options like a `program_id` or `group_id`.	write-only embed
initial_enrollment.program_id	integer	The program and the term both have a `term_type` attribute; this value must equal or the `program_id` will be invalid.	write-only
initial_enrollment.group_id	integer	Assign the student to a group; it should be associated with the term: `group.term_id` should match `enrollment.term_id`	write-only
initial_enrollment.grade_level	integer	Default: 0. Set the initial enrollment grade level.	write-only
enrollment_id	integer	When using POST or PUT, add this field in the `include_fields` to have access to the `initial_enrollment` ID. ⚠️ Using the query param `dry_run=true` will not generate an `enrollment_id`. When using GET to list students by course or term, this field will be the enrollment ID that corresponds to the course/term respectively.	read-only extra
enrollments	Enrollment	When using GET to fetch a single user, this field contains all the related enrollments in descending order by date. Not available when using GET to list multiple students, use GET enrollments instead. **Field available in this priority:** • GET List: not available. • Admin User: available. • Other User: not available.	read-only extra has-many
ad_campaign	object	Connect an advertising campaign with the student. This attribute is an object because you can attach a small description. **Field available in this priority:** • GET List: not available. • Admin User: available. • Other User: extra field.	embed

Attribute	Type	Description	Notes
ad_campaign.id	AdCampaign	Use an existing `id` from the AdCampaing catalog.	belongs-to
ad_campaign.custom_description	string	Complement the ad campaign with a description related only to the student.	
custom_fields	array	Saeko can store a set of custom key-value objects. E.g. `[{key: "interests", value: "sports,movies"}]` When updating a student you can only update or set new custom fields. If you want to remove a field PUT a `{ key: "key_to_remove", value: null }`. **Field available in this priority:** • GET List: not available. • Admin User: available. • Other User: not available.	embed extra
custom_fields[].key	string	**MUST** start with a lower case letter and must match `/[a-zA-Z0-9_]/` E.g. "interest1", "previous_school", "recommendedBy" Any other invalid character will be transliterated or removed, e.g. `"1año-Estadía2" => "anoEstadia2"`	
custom_fields[].value	*	Supported value types: **string**, **number** and **boolean**. Any other value type will be consired `null`. Use a `null` value to remove a field from the `custom_fields` array.	

LIST OF ENDPOINTS

- GET /api/v1/core/terms/term_id/students

  List ACTIVE students filtered by `term_id`.
  The response objects contain a small set of fields; if more fields are required then use the GET student by id request.

  Allowed Users: admin
- GET /api/v1/core/students/id

  Show student details.
  `include_fields` supported for extra fields.

  Allowed Users: adminprofessor
- GET /api/v1/core/student

  Show the student if the current user is a student.

  Allowed Users: student
- POST /api/v1/core/terms/term_id/students

  Create a new student with an initial enrollment in the `term_id`.
  Query Params:

  - `fields` `include_fields` - supported for extra fields.
  - `dry_run` - when `true` then the student object will be validated but not saved.
    - The `id` response will be `null`.
    - The `student_id` cannot be auto-generated.

  Allowed Users: admin
- PUT /api/v1/core/students/id

  Update an existing student.
  `include_fields` supported for extra fields.

  Allowed Users: admin

</> Employee

DESCRIPTION

En employee is a person who belongs to one or more schools, and may have a role of Adminsitrator or Professor. To create a Professor just create an Employee with professor details. To create an Adminsitrator just create an Employee with the administrator role.

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
name	string		required
first_surname	string		required
second_surname	string		
gender	string	`MALE = 1` `FEMALE = 2`	required
marital_status	integer	`SOLTERO = 1` `CASADO = 2` `DIVORCIADO = 3` `VIUDO = 4` `UNION_LIBRE = 5` `SEPARADO = 6`	
birthplace	string		

Attribute	Type	Description	Notes
birthdate	date		
curp	string		
rfc	string		
personal_email	string		
blood_type	string		
payroll	string	Nómina, número de registro, etc.	
cost_per_hour	string	Hourly wage	
professor_details	object	Use this object when the employee is a professor. This will create a professor that can be used later to associate courses.	
professor_details.create	boolean	It is only required when creating a professor the first time. It is optional on futures updates.	
professor_details.comments	string		
professor_details.professional_license	string		
professor_details.date_admission	date		
professor_details.dictum	string		
professor_details.grade_level	string		
professor_details.master_degree	string		
professor_details.doctors_degree	string		
professor_details.sep_authorization_number	string		
professor_details.payroll	string	The employee object and professor object may have different payroll numbers. Use the same if it is the case.	
administrator	Administrator	Use this object when the employee is an adminsitrator.	has-one
administrator.role_id	integer	`ROLE_ADMINISTRATIVO_ID = 1` -- super admin `ROLE_PERSONALIZADO_ID = 2` -- custom permissions by school `ROLE_COLEGIO_ID = 4` -- custom permissions by group of schools	required
administrator.position	string		
administrator.password_v1	string	To grant access to Saeko v1 this password can be set.	
phones	Phone		has-many
addresses	Address		has-many
medical_details	MedicalDetail		has-many
occupations	Occupation		has-many
school_ids	array		required

LIST OF ENDPOINTS

- GET /api/v1/core/schools/school_id/employees

  List of employees that belongs to a school

  Allowed Users: admin
- GET /api/v1/core/employees/id

  Show employee by ID.

  Allowed Users: admin
- POST /api/v1/core/employees

  Create a new employee

  Allowed Users: admin

Contact

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
type	string	`student`, `professor`, `admin`, `relative`, `employee`, `school`	read-only
name	string		read-only
surname	string		read-only
second_surname	string		read-only
gender	integer	`MALE = 1`, `FEMALE = 2`	read-only
avatar_url	string		read-only

/api/v1/location

Address

DESCRIPTION

Saeko have a static database of all official places in Mexico.

In case you want to create an address with a custom suburb or city, use the `suburb_name` or `city_name` attribute instead.

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
type	integer	`HOME = 1`	
street	string		
street_number	string	House number, apartment number, etc. Limit: 20 chars.	
zipcode	string	Must be 5 characters or an empty/null value	
suburb_id	Suburb		belongs-to
suburb_name	string	This will NOT create a new suburb, but just save a custom suburb name for this address.	
city_id	City		belongs-to
city_name	string	Custom city name for this address	
state_id	State	A State ID from the location database	required belongs-to

State

DESCRIPTION

Saeko have a static database of all official cities in Mexico.

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
name	string		

LIST OF ENDPOINTS

- GET /location/countries/country_id/states

  List of states by Country

  Allowed Users: any

Country

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
name	string		

LIST OF ENDPOINTS

- GET /location/countries

  List of available countries

  Allowed Users: any

/api/v1/accounting

Scholarship

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
scholarship_type_id	ScholarshipType	Every scholarship must belong to a ScholarshipType	required belongs-to
amount	decimal	This amount can be a percentage (e.g. 30%) or a fixed number (e.g. 100.0); two-digit decimal allowed.	required
amount_type	integer	`PERCENTAGE: 1` `FIXED: 2`	required
applicable_installment_type	integer	Saeko can apply the discount on the first/initial installments, partial/recurring installments, or both. `FIRST_INSTALLMENTS = 1` `PARTIAL_INSTALLMENTS = 2` `ALL_INSTALLMENTS = 3`	
comments	string		

LIST OF ENDPOINTS

- GET /api/v1/accounting/enrollments/enrollment_id/scholarship

  Get the enrollment `belongs-to` scholarship association object.

  Allowed Users: admin
- POST /api/v1/accounting/enrollments/enrollment_id/scholarship

  Create a scholarship for the enrollment. Updates in case the enrollment has an existing scholarship record.

  Allowed Users: admin

ScholarshipType

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
name	string		

LIST OF ENDPOINTS

- GET /api/v1/accounting/scholarship_types

  List all available scholarship types; required when creating Scholarships.

  Allowed Users: admin

/api/v1/admissions

AdCampaign

DESCRIPTION

Basic catalog of advertising campaigns.

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
name	string		

LIST OF ENDPOINTS

- GET /api/v1/admissions/ad_campaigns

  List of ALL ad campaigns

  Allowed Users: admin

/api/v1/grading

Activity

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
name	string		
description	string		
due_date	datetime		
activity_type	integer	`TIPO_GENERAL = 1` `TIPO_PERIODO = 2` `TIPO_FINAL = 3`	
grade_mode	integer	Child activities will calulcate the parent grading using one of these calculations: `MODO_PROMEDIO = 1` -- average of all available scores `MODO_SUMATORIA = 2` -- sum of all scores `MODO_PERSONALIZADO = 3` -- using custom weights `MODO_NINGUNO = 4` -- none	
grade_type	integer	`GRADE_SCORE = 1` `GRADE_PERCENT = 2`	
weight	decimal	Required when using `grade_mode: MODO_PERSONALIZADO`.	
editable	boolean	TRUE when this activity can be modified by the current user.	read-only
gradable	boolean	TRUE when this activity can be graded by the current user under various circumstances.	read-only
locked	boolean	TRUE when this activity will not calculate scores when children scores change.	read-only
deletable	boolean	TRUE when this activity can be deletable by the current user.	read-only
parent_id	Activity		belongs-to
replace_id	Activity	When grading an activity that has a replace_id, the parent score will be calculated using this activity instead of the replace_id activity.	belongs-to
course_id	Course		read-only belongs-to

Attribute	Type	Description	Notes
periodo_id	Period	Only available when `activity_type: TIPO_PERIODO`	read-only belongs-to

LIST OF ENDPOINTS

- GET /api/v1/grading/courses/course_id/activities

  List of grading activities by `course_id`.

  Allowed Users: adminprofessorstudent

Grade

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
score	string	This can be a letter mark score or a decimal number. It will always be validated using various institution rules.	
activity_id	Activity		read-only belongs-to
student_id	Student		belongs-to
published_score	string	When the activity is `activity_type: TIPO_PERIODO` or `activity_type: TIPO_FINAL`, the score must go through another set of rule so it can be piublically visible to students and parents, as well as official report cards and other reports. In practice, calculations on child activities can be done automatically without pubishing the results, this value can be found in the `score` field; then the professor must review the calculated scores, and publish, making modifications on the way if required. If the conditions are met the normal `score` will be automatically published, that is why this field is readonly.	read-only

LIST OF ENDPOINTS

- GET /api/v1/grading/courses/course_id/grades

  List of grades by `course_id`. This includes grades for all students in all activities in the `course_id`.

  Allowed Users: adminprofessor
- GET /api/v1/grading/activities/activity_id/grades

  List of grades by `activity_id`. Smaller set of grade of all students of a single activity from a single course.

  Allowed Users: adminprofessor
- POST /api/v1/grading/activities/activity_id/publish

  Save grades by `activity_id`.

  Allowed Users: adminprofessor

  Example:

```
{
  "grades": [
    { "score": 10, "student_id": 16 }
    { "score": 9.1, "student_id": 17 }
    { "score": 5, "student_id": 18 }
  ]
}
```

ExtraordinaryGrade

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer	Use this attribute to update existing extraordinary grades with the POST endpoint.	read-only
score	string	This can be a letter mark score or a decimal number. It will always be validated using various institution rules. Do no send this attribute when creating a `grading_ext_request` (Actas Extraordinarias)	required
type	integer	`EXT = 1` `ETS = 2` `CV = 3` `CR = 4` `EQ = 5` `RS = 6` `INT = 7` `PAS = 8`	required
date	date	Because a student can have multiple extraordinay grades on the same course, Saeko will use the date to sort and prioritize the most recent records.	required

LIST OF ENDPOINTS

- GET /api/v1/grading/courses/courses_id/extraordinary_grades

  List of extraordinay grades by `course_id`.

  Allowed Users: admin
- GET /api/v1/grading/students/student_id/extraordinary_grades

  List of extraordinay grades by `student_id`.

  Allowed Users: admin
- POST /api/v1/grading/courses/course_id/extraordinary_grades

  Save one or multiple extraordinay grades by `course_id`.
  When sending an array of extraordinary grades, is possible to update an existing record by sending his ID.

  Allowed Users: admin
  Example:

```
{
  "extraordinary_grades": [
    { "score": "90", "student_id": 8023, "type": 1, "date": "2020-06-01" }
    { "id": 1234, "score": "80" }
  ]
}
```

- DELETE /api/v1/grading/courses/courses_id/extraordinary_grades/:id

  Remove an existing extraordinary grade.

  Allowed Users: admin

/api/v1/scheduling

GateAccessLog

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
card_number	string	A string representing the card of the person	required
direction	integer	Optionally create a record with IN or OUT access: `IN = 1` `OUT = 2` -- when recording OUT access logs, Saeko will try to mutually associate the respective IN access log made on the same day.	
recorded_at	datetime		read-only
contact	Contact		read-only belongs-to
related_gate_access_log	GateAccessLog	When creating an OUT access log this embed resource may be available.	read-only belongs-to

LIST OF ENDPOINTS

- POST /api/v1/scheduling/gate_access_logs

  Record when a person access the gate using a card number.

  Allowed Users: admin
  Example:

```
{
  "gate_access_log": {
    "card_number": "123456789",
    "direction": 1
  }
}
```

/api/v1/provisioning

User

DESCRIPTION

Students, professors, administrators and relatives can have credentials to access their respective portals.

These users can be created on external provisioning services: Google Suite or Microsoft Azure. **(Special setup required)**

When using the default provisioning services, user accounts are managed by Saeko and users must confirm the email address and setup a password.

MODEL DEFINITION

Attribute	Type	Description	Notes
id	integer		read-only
email	string		required
reclaim_email	boolean	This field is very important when the email exists in the external provider (Google Suite or Microsoft Azure) and Saeko should only link (**reclaim**) the existing email to the contact (e.g. profesor, student). Using this field will not reset the account's password, neither send the user any email instructions for password setup. The user can login using one of the SSO buttons: Sign in with Google or Sign in with Microsoft.	

LIST OF ENDPOINTS

- POST /api/v1/provisioning/contactable_type/:contactable_id/user

  Create a user by contactable type: `students`, `professors`, `relatives`, `administrators`.

  E.g. `POST /api/v1/provisioning/students/1/user`

  E.g. `POST /api/v1/provisioning/professors/1/user`

  Allowed Users: admin